Kubron

Helm Module: User Guide & Developer Documentation

Version 0.2.0 · Wails + Vue + Go · October 2026

Part I. User Guide

1. Overview

The Helm module in Kubron provides full management of Helm releases from the desktop client: list, install, upgrade values and chart version, rollback, and uninstall. It talks to the Kubernetes API (Secrets/ConfigMaps that store releases) and uses the Helm SDK — not the `helm` CLI on the user's machine.

Key capabilities:

- Release list with hints when a newer chart version is available
- Chart catalog: search, README, values, install with dry-run
- Repository management (HTTP and OCI, basic auth, skip TLS)
- Upgrade values or chart version with a manifest diff preview
- Revision history, diff, and rollback
- Live operation progress with cancel (install / upgrade)
- Journal of the last 200 operations with Helm logs
- Secret (default) and ConfigMap release storage

ℹ️ Chart repositories are stored locally under ~/.kubron/helm and are independent of the cluster. Releases are read from the active cluster.

2. Navigation

Under Helm in the sidebar:

- Releases — installed releases in the current (or selected) namespaces
- Catalog — charts from configured repositories
- Repositories — chart sources
- Operations — operation history

The cluster dashboard Helm card shows a summary: release count, failed/pending, and update hints.

3. Repositories (Helm → Repositories)

Chart sources are stored locally at `~/.kubron/helm`. They are not tied to a cluster.

3.1. Adding a repository

1. Click "+ Add".
2. Enter a name (e.g. bitnami) and URL.
3. Optionally set username/password (basic auth). The password is stored with mode 0600.
4. For an OCI registry, use an oci://… URL and list the chart names (comma-separated). OCI has no chart index.
5. Optionally enable "Skip TLS verification".

6. Click Add. Kubron checks reachability and refreshes the index.

ℹ️ Insecure HTTP for OCI is allowed only for localhost.

3.2. Updating indexes

- “Update all” refreshes every repository index.
- Each repository has its own Update action.

For OCI, versions come from image tags.

3.3. Removing

Removing a repository does not affect releases already installed from it.

4. Chart catalog (Helm → Catalog)

Search by name, description, or keywords. Filter by repository. Columns: chart, repository, version, app version, description. Deprecated charts are labeled.

4.1. Chart detail

- README
- Values — default values.yaml
- Info — website, sources, maintainers, keywords, versions
- Install opens the install dialog

5. Installing a release

From the catalog: open a chart → Install.

5.1. Parameters

- Release name — required; Kubernetes naming rules (lowercase letters, digits, “-”, max 53 chars)
- Namespace — existing or new (create if missing)
- Chart version
- Release storage: Secret (default, same as helm CLI) or ConfigMap
- Wait for resources ready (up to 5 minutes) — blocks the dialog while waiting
- Values: your overrides on the left, chart defaults on the right (reference only)

5.2. Preview (dry-run)

Preview renders the manifest without touching the cluster. Template lookup functions return empty values; Secret data is masked. Notes (NOTES.txt) are shown when present.

5.3. Execution

After Install, a live progress panel appears. Install can be cancelled. On success the release appears under Helm → Releases.

⚠️ Install needs write permission and usually access to secrets in the target namespace.

6. Releases (Helm → Releases)

Lists the latest revision of each release. Columns: name, namespace, chart, chart version, app version, revision, status, updated, storage. If a newer chart version exists in your repositories, an update hint is shown.

6.1. Release detail

Open a release for tabs:

- Content — user-supplied / all values (with defaults), Notes, copy
- History — revisions, diff, rollback
- Resources — objects from the release manifest
- Upgrade (Edit values) — change values only
- Chart version — change the chart version

7. History and rollback

The History tab lists all revisions. The current one is marked. You can diff any pair (manifest / values).

Rollback: pick a revision → Rollback. A new revision is created with that content. Confirmation is required.

ℹ️ Cancel is not available for rollback with the current Helm SDK integration — the operation runs to completion.

8. Upgrade values (Edit values)

Only values change; the chart version stays the same.

- Left: current user values; right: editable copy
- Deleting a key restores the chart default
- Preview changes — manifest diff (Secret values masked)
- If the release changed while you edited (stale base revision), a warning and reload action appear

Apply creates a new revision. Live progress and cancel are available.

9. Chart version upgrade

Moves the release to another version of the same chart. Requires write permission and secrets access.

- Pick the repository (if the chart is found in more than one)
- Target version (optional prereleases)
- Labels: current / prerelease / deprecated
- Downgrades are flagged — prefer History → Rollback for a safe revert
- User values are carried over; everything else comes from the new chart defaults
- Manifest diff preview, then Apply chart upgrade

ℹ️ If the chart is not found in any repository, add the repo under Helm → Repositories and refresh the index.

10. Uninstall

From the release list or detail — Uninstall, with confirmation. Live progress is shown; cancel is not available for uninstall in the current SDK integration.

11. Operations journal (Helm → Operations)

History of installs, values upgrades, chart upgrades, rollbacks, and uninstalls. Last 200 operations are kept, each with a Helm log.

- Filters: operation type, status (success / failed / canceled), search by release, namespace, cluster, error text
- In-progress operations are highlighted
- Clearing the journal does not touch the audit log

In web mode an operation is bound to the owner session; only the owner can cancel it.

12. Permissions and roles

Local Kubron roles:

Role	Helm access
admin	Full: install, upgrade, chart upgrade, rollback, uninstall, repositories
operator	Release operations (write / secrets); repository management per policy
user	View releases, history, catalog (read)

Cluster RBAC still applies: without list access to secrets/configmaps in a namespace, the release list may be incomplete or empty.

13. Release storage

By default Helm stores revisions in Secrets of type `helm.sh/release.v1` (label owner=helm). Alternative: ConfigMap (HELM_DRIVER=configmap). Kubron reads both via the API and lets you choose the type at install time.

Payload: base64(gzip(JSON)). Superseded revisions are omitted from the main list but remain available in History.

14. Progress and cancel

Install and upgrade (values / chart version) show a progress panel with a log and can be cancelled. Rollback and uninstall do not support cancel in the current Helm SDK integration; they run until completion (timeout 5 minutes).

15. Practical tips

7. Add the repositories you need (bitnami, prometheus-community, your OCI registries) and refresh indexes first.
8. For production installs, use dry-run and read the Notes carefully.
9. After a bad upgrade, prefer History → Rollback over a chart downgrade.
10. If a release “disappears”, check the namespace filter and RBAC on secrets/configmaps.
11. Update hints appear only when the matching repository is added and its index is current.
12. In multi-cluster mode repositories are shared (local); releases are scoped to the active cluster.

16. Limitations in 0.2.0

- Cancel is available only for install and upgrade

- Helm plugins and post-renderers are not supported yet
- OCI: chart names must be listed explicitly
- Web mode: Helm UI is wired, but the web product is still under active development
- No built-in lint / template validate beyond install dry-run

Part II. Developer Documentation

17. Architecture

Helm in Kubron is not a CLI wrapper. It integrates via `helm.sh/helm/v4` SDK plus direct Secret/ConfigMap reads through client-go.

17.1. Layers

Layer	Responsibility
Frontend (Vue 3)	Views, tabs, composables, resource definitions, i18n
Wails bridge	callApp('MethodName', ...) → Go Manager methods
internal/k8s (Go)	Helm SDK actions, store, repo, progress, journal
Kubernetes API	Secrets/ConfigMaps (releases), REST for apply
Local filesystem	~/.kubron/helm — repos, cache, operations.json

17.2. Backend files (internal/k8s/)

File	Purpose
helm.go	GetHelmReleases / GetHelmCharts, list payloads
helm_store.go	Read Secret + ConfigMap, decode base64/gzip/JSON
helm_actions.go	Rollback, Uninstall, Diff, helmConfig*
helm_install.go	Install + dry-run, HelmInstallRequest/Result
helm_upgrade.go	Upgrade values only
helm_chart_upgrade.go	Chart version change (v0.2.0)
helm_detail.go	Content (values/notes/manifest) by revision
helm_repo.go	HTTP/OCI repositories, index, search, package cache
helm_oci.go	OCI registry: tags, pull chart
helm_updates.go	Find newer chart versions in local indexes
helm_progress.go	opID, tracker, cancel, helm:progress event
helm_journal.go	operations.json, last 200 records
helm_cache.go	Informer / helm:changed event

Each file has a matching `*_test.go`.

17.3. Frontend

- `frontend/src/components/views/` — HelmReposView, HelmCatalogView, HelmInstallDialog, HelmOperationsView
- `frontend/src/components/resource/sections/` — HelmContentTab, HelmHistoryTab, HelmResourcesTab, HelmUpgradeTab, HelmChartUpgradeTab
- `frontend/src/composables/useHelmOps.ts` — progress, cancel, runHelmOp, helmJournalTick
- `frontend/src/composables/useHelmLive.ts` — helmLiveTick on helm:changed

- `frontend/src/resources/definitions/helmrelease.ts` — defineResource: columns, tabs, actions (uninstall)
- `frontend/src/resources/definitions/helmchart.ts` — aggregation by chart
- `frontend/src/i18n/locales/{en,ru}/helm.json` — all UI strings

18. Data model (Go)

HelmReleaseInfo

Latest revision for the list: name, namespace, chart, chartVersion, appVersion, revision, status, updated, description, storage (secret|configmap), latestVersion / latestRepo / latestDeprecated.

HelmInstallRequest

repo, chart, version, releaseName, namespace, values (YAML overrides), createNamespace, wait, storage, opId.

HelmInstallResult

releaseName, namespace, revision, status, manifest (secrets masked), notes.

HelmRevisionDiff

from, to, values (unified diff), manifest (unified diff, secrets masked).

HelmOperationRecord

id, op, cluster, namespace, release, detail, revision, status (success|failed|canceled), error, startedAt, finishedAt, durationMs, log[].

HelmRepoInfo

name, url, username, insecureTLS, charts, updated, error, kind (http|oci), chartNames (OCI only).

19. Release storage (helm_store.go)

Object name: `sh.helm.release.v1.<name>.v<N>`, label `owner=helm`, field `data.release = base64(gzip(JSON))`.

- Secret type: helm.sh/release.v1
- ConfigMap: no type; recognition by release field + label
- listHelmStored reads both; Forbidden on one store is tolerated if the other succeeds
- decode: base64 → gzip (LimitReader 64 MiB) → JSON
- Mutating operations use the same driver where the release lives — otherwise Helm cannot find it

Constants: `helmStorageSecret` = "secret", `helmStorageConfigMap` = "configmap".

20. Repositories (helm_repo.go)

Directory: `~/.kubron/helm/` (not ~/.config/helm).

- `repositories.yaml` — Helm repo.File format, mode 0600 (passwords)
- `cache/<name>-index.yaml` — downloaded indexes
- Repository name is strictly validated (regex); it is part of the file name
- Credentials are sent only to the repository host (not to redirects or chart URLs from the index)
- Limits: index 64 MiB, chart archive 32 MiB, HTTP timeout 30s, parallel update 4

- In-memory index cache and LRU package cache (16)

OCI (helm_oci.go): no index.yaml; versions = tags; chart names are user-provided.

21. Operations and progress (helm_progress.go)

Flow:

13. Frontend generates opId (UUID / [A-Za-z0-9-]{8,64}) before calling the backend.
14. Backend: beginHelmOp(ctx, opID, op, ns, rel, cancelable) → tracker + optional cancel context.
15. Event
16. Log lines batched (≤ 40), tick 250 ms, heartbeat 1 s.
17. CancelHelmOperation(opId) calls tracker.cancel (only if cancelable=true).
18. finish() writes a journal record and emits done=true.

Cancelable: install and upgrade — yes (action context). Rollback and uninstall — no (current Helm SDK actions do not take a cancellable context); timeout 5 minutes.

Frontend (useHelmOps.ts):

- Subscribe via onEvent('helm:progress')
- helmOps ref — active and recently finished ops
- Successful ops auto-dismiss after 6 s
- helmJournalTick++ on done → Operations page reloads history
- runHelmOp({ op, namespace, release, cancelable }, fn) wrapper

ℹ️ In web mode an operation is bound to the owner session; only the owner can cancel it.

22. Live updates (helm_cache.go + useHelmLive)

Informer on Secrets (and ConfigMaps when possible) with label owner=helm. On change — EventsEmit `helm:changed`.

useHelmLive: helmLiveTick++. The helmrelease resource definition refreshes via refreshOn / autoRefresh. If the user cannot list secrets, the informer does not start and polling remains.

23. Operations journal (helm_journal.go)

File: `~/.kubron/helm/operations.json` (0600), shared across clusters; each record includes cluster.

- Max 200 operations, up to 80 log lines each
- Operational log, not audit: “who” is recorded in the audit log
- GetHelmOperations(query), GetHelmOperation(id), ClearHelmOperations()

24. Resource definition (frontend)

helmrelease: defineResource with namespaced: false (aggregate), editable: false (no object YAML editor).

Tabs: values / notes / manifest (helmContentTab), editValues (HelmUpgradeTab), upgradeChart (HelmChartUpgradeTab), resources, history.

Actions: uninstall (Perm.Write, danger, confirm → runHelmOp).

Update column: latestVersion from backend (helm_updates.go + local repo indexes).

25. Permissions and security

- Local Perm (Read / Write / Secrets / …) checked in UI and backend
- GetHelmReleaseContent / Diff require Secrets — values may contain passwords
- Secret values are masked in manifest/diff before reaching the UI
- Repository passwords: mode 0600, not logged, sent only to the repo host
- validateHelmRepoName / validateHelmRef / chartutil.ValidateReleaseName — before any network I/O
- Gzip bomb protection: LimitReader 64 MiB on decode

26. Dependencies

- `helm.sh/helm/v4` — action, chart, repo, kube, release
- `k8s.io/client-go` / cli-runtime — REST, ConfigFlags, informers
- Wails EventsEmit / EventsOn for progress and live updates

Helm v4: action.Configuration.Init(flags, namespace, driver), WaitStrategy = kube.StatusWatcherStrategy.

27. Adding a new Helm operation

19. Backend: Manager method, beginHelmOp(..., cancelable), defer tr.finish(err), use helmConfigForRelease / helmConfigDriver.
20. For progress logs, pass tr.handler() to SetLogger before Init.
21. Mask secrets in any returned manifest.
22. Frontend: runHelmOp({ op, namespace, release, cancelable }, () => callApp('Method', ...)).
23. Add i18n keys in en/ru helm.json (ops_panel, operations).
24. Test: *_test.go with fake client / httptest for repos.

28. Tests

All helm_*.go files have table-driven tests. Typical cases:

- Decode of corrupted / oversized release payloads
- List with Forbidden on secrets but OK on configmaps (and vice versa)
- Rollback/upgrade with the wrong storage driver
- Repo name validation, index size limits
- opID validation, cancel race
- Journal trim to 200

29. Related product areas

- Dashboard — Helm card (HelmCard.vue)
- Argo CD — separate module (GitOps); do not mix with Helm releases
- Audit log — who ran operations (local users)
- pkg/ext — extension points (SSO, etc.), not Helm-specific

This document is based on Kubron v0.2.0 source and UI (https://github.com/VadimII224/kubron). If it disagrees with the code, the repository wins.